Individual Reflective Report: Autonomous SAR Drone Project

Dmytro Chuprynyuk

AENGM0074 Group Design Project — University of Bristol — March 2026

1 Introduction

I was the software and computer vision lead for an autonomous SAR drone (AENGM0074 Group Design Project), responsible for the full software stack: state machine, vision system, search planning, target localisation, ground station, and testing infrastructure (∼15,000 lines of Python). This report examines my contributions through AHEP4 learning outcomes [1]: design and problem-solving (M5), societal and environmental impact (M7), and teamwork and communication (M16, M17).

2 Design and Problem-Solving (M5)

2.1 Original Solutions to Complex Problems

The core challenge was building a fully autonomous SAR mission on a Raspberry Pi 5 [2] with no GPU, operating at under 5 FPS, where software failures have physical consequences. This constraint turned out to be generative: it forced design decisions that a more powerful platform would have made unnecessary, and taught me to treat limitations as design drivers rather than obstacles.

Dual-backend vision architecture. The vision system exposes a single interface—`detect_in_image(frame)`—while auto-detecting the platform and selecting the appropriate backend (Ultralytics on laptop, TFLite [3] on Pi). This abstraction let me add lens undistortion, NCNN acceleration, and model swapping without changing any other module. When Edward benchmarked detection thresholds and calibrated the camera's field of view, those parameters slotted into vision.py without any integration pain—a validation of the modular boundary I had drawn. Economically, the £60 Pi platform makes the entire system accessible to volunteer SAR teams and resource-limited organisations who cannot afford GPU-equipped drones, a consideration that shaped every hardware choice.

Simulation-first development. Following Schön's reflection-in-action principle [4], I built an interactive simulator with configurable GPS drift (up to 3 m), camera shake, and inference throttling, proving every algorithm under pessimistic conditions before deployment. This approach replaced approximately 50 physical test flights, each consuming a LiPo charge cycle and carrying crash risk—an environmental and safety benefit that emerged from a technical decision. Legally, simulation also let me validate geofencing logic against the SSSI no-fly zone boundary without risking an actual airspace violation under CAA regulations.

Target localisation under uncertainty. Converting pixel detections to GPS coordinates required fusing camera geometry with drone telemetry via inverse-variance weighting ($w = 1/h^2$), with a bonus for near-centre detections where projection error is minimised. Video analysis of DJI flight footage revealed GPS timing lag (100–200 ms), causing ∼1 m directional error at 5 m/s. Rather than adding complexity with a compensation filter, I relied on statistical averaging across multiple passes—a conscious trade-off between accuracy and system simplicity that Leveson's work on safety-critical design [5] reinforced for me: in a system this close to hardware, every line of code is a potential failure point.

2.2 Safety-Critical Design

Working on a system that controls a physical aircraft shaped every design decision. I designed a five-step progressive testing sequence—from Mission Planner waypoints (no custom code) through passive CV logging to full autonomy—following aerospace verification practices [6]. Each step adds exactly one variable, making failure causes unambiguous. The human-in-the-loop verification stage exists because I believe autonomous systems should augment human judgement, not replace it—particularly when the "target" is a potentially injured person. The RC kill switch, giving the pilot absolute priority over any software command, reflects the same principle.

2.3 Diversity and Inclusion

Our team of five spanned five nationalities, which meant technical discussions naturally surfaced different assumptions about documentation, testing rigour, and interface design. I built the ground station as a browser-based web application partly because teammates had different operating systems, but also because it makes the system usable by SAR volunteers who may not be technically trained. All 16 documentation files were written in English as the team's lingua franca, avoiding jargon in user-facing guides. This experience taught me that inclusive design is a natural consequence of working in a diverse team: when you cannot assume a shared background, you design for the general case, which benefits everyone.

3 Societal and Environmental Impact (M7)

3.1 Positive Potential

SAR drones reduce search time by covering terrain faster than ground teams and detecting casualties in areas too dangerous or remote for human access [7, 8]. What makes *our* system's contribution specific is the cost barrier it removes: a £60 Raspberry Pi running a 3.2 MB TFLite model makes autonomous visual search viable for volunteer mountain rescue teams, community first responders, and organisations in countries where helicopter searches are unaffordable. The browser-based ground station requires no specialist software—any phone or tablet with a browser becomes a command interface—which matters when the operator may be a volunteer, not an engineer. I designed these choices deliberately, not just for technical elegance but because I believe the gap between "technology exists" and "technology is accessible" is where engineering effort is most needed.

3.2 Environmental Impact

I made deliberate choices to minimise the project's environmental footprint. Simulation-first development replaced approximately 50 physical test flights, avoiding an estimated 125 kg CO_2 equivalent (each flight consumes a 6S LiPo cycle with associated battery amortisation, motor wear, and travel). The Pi 5 consumes ∼15 W under inference versus 300 W+ for a GPU server; over a 20-minute mission, 5 Wh versus 100 Wh. Model training used Google Colab's shared GPU (∼0.5 kWh for 150 epochs). YOLOv8n was chosen as the smallest viable architecture, and automatic SSSI geofencing protects the site's rare bird habitats. The drone platform is reusable for future cohorts, and the software is MIT-licensed. A full lifecycle assessment would need to account for rare-earth minerals in the flight controller and lithium battery recycling.

3.3 Dual-Use and Adverse Impact

The same technology that searches for a missing person can surveil one. I am not able to discuss this abstractly: as a Ukrainian student, the military application of autonomous surveillance drones is not a hypothetical scenario for me. I have seen what these systems do when the "target" is not a casualty to rescue but a person to track. This personal experience made me deliberate about every design choice that touches autonomy.

The system I built detects and localises, but a human operator decides what to do—human-in-the-loop verification is not a feature I added for the rubric, but a constraint I insisted on because I understand what happens when it is removed. Images are processed locally and never transmitted to a cloud service; detection logs are stored on the Pi's SD card with no persistent surveillance capability. The confidence threshold (0.4) was set conservatively to reduce false positives, because in SAR, a false detection wastes search time that a real casualty cannot afford.

However, I must be honest about the limits of these safeguards. The human-in-the-loop is a software constraint, not a hardware one—removing the verification prompt is a single-line code change. The MIT licence means anyone can fork and strip the geofence and verification. The edge AI architecture avoids cloud dependency but also means no external oversight. If I were to commercialise this work, I would explore hardware-enforced geofencing (e.g., a GPS-locked relay) and code-signing to prevent modified firmware—moving safety from software policy to physical constraint [5].

4 Teamwork, Leadership, and Communication (M16, M17)

Our five-person team had distinct roles: I led software and CV, others handled hardware assembly, piloting, project administration, and flight dynamics research. In Belbin's framework [9], I functioned as Specialist and Completer-Finisher—technical lead and de facto systems integrator. The team worked effectively where responsibilities had explicit handoff points: the hardware lead assembled the drone independently, consulting me only for Pi-to-Cube wiring; the pilot practised RC and understood abort procedures without engaging with code; the project manager booked field slots and submitted risk assessments I would have neglected.

The most productive sub-team pairing was with Edward on computer vision. His optics knowledge complemented my software skills—he calibrated camera FOV and evaluated lens distortion while I built the detection pipeline. This division by complementary expertise produced faster results than either of us working alone.

In practice, the broader software was written by one person. This evolved because I had the strongest programming background and found it faster to build than onboard. Reflecting through Kolb's cycle [10], I had optimised for output over team development. I attempted to lower the barrier with numbered test scripts and documentation, but the combined context of MAVLink, state machines, and TFLite was too high for short-term engagement. Teammates expressed frustration at feeling sidelined—the most persistent tension in the team, and I bear primary responsibility. The lesson: contribution points must be designed in from the start, not retrofitted.

Disagreements and compromise. The first technical disagreement—ROS 2 versus raw MAVLink—I resolved by building a 20-line working heartbeat demo. In Tuckman's model [11], this storming phase was brief because the team deferred to demonstrated results. However, this was persuasion by fait accompli, not negotiation. The second disagreement taught me more. I had designed a state machine with 11 states and multiple sub-modes; the team pushed back, arguing it was too complex for anyone else to debug under pressure. After sitting with their feedback, I recognised they were right about

the maintainability risk. I simplified the state transitions, removed two experimental sub-modes, and wrote explicit transition diagrams. This compromise cost a week of refactoring, but produced a system the whole team could reason about on flight day. It was the first time I genuinely changed a technical direction because of team input rather than technical evidence, shifting my view of "good architecture" from "what I can debug" to "what the team can debug."

Receiving and acting on feedback. The pilot identified that abort and confirm-target buttons in the ground station were dangerously close—a genuine safety risk I had missed. I immediately separated the controls and added a confirmation step. The project manager told me my documentation was too dense; I restructured key guides around numbered steps and created a one-page colleague checklist. Edward noted that my iteration pace made it hard to track which module version was current; I responded with explicit version tags and a living ground-truth document. Following Gibbs [12], these exchanges taught me that designing for external feedback is itself an engineering discipline.

Communication across skill gaps. The **simulator** proved the most effective communication device: teammates understood the system intuitively when they saw the drone flying, detecting, and descending on screen. The **ground station** translated complex state into large buttons usable outdoors. For the pilot, I ran through the abort sequence on a bench test until he could make independent decisions during flight. On the cancelled flight day, he navigated the ground station independently—confirming targets and reading telemetry without verbal guidance—the first evidence the interface worked as intended.

Team effectiveness evaluation. The team delivered a working system, but the five-person structure was suboptimal for a software-heavy project. Three roles had low sustained workload, while software demanded 25–30 hours per week from me alone. A more effective allocation would have paired two people on software from day one with explicit interface contracts and code review cycles.

5 Self-Assessment and Development

5.1 Skills Developed

This project pushed me into unfamiliar domains. I learned MAVLink and ArduPilot [13, 14] from scratch—heartbeat negotiation, command acknowledgement, and autopilot configuration. Edge AI deployment (model export, TFLite [3] post-processing, Pi benchmarking) taught me that infrastructure around a model is where real complexity lies. Threading camera capture, inference, telemetry, and web streaming on a single-core Python process gave practical concurrency experience under GIL constraints. Field debugging on a Pi over SSH with no internet and a PuTTY terminal was perhaps the most underrated skill developed.

5.2 Time Management and Self-Management Failures

I invested approximately 25–30 hours per week across weeks 12–20, compared to an estimated 8–12 hours from most teammates—partly by necessity, partly by choice (Section 4). Work was structured around weekly milestones: weeks 12–15 on simulation, 16–18 on Pi deployment, 19–20 on field testing and retraining. A living ground-truth document (`CLAUDE.md`, 800+ lines) tracked every session's progress. Version control discipline—feature branches, progressive commits, never breaking `main`—provided both a safety net and an audit trail.

Where self-management failed. I underestimated Pi integration by at least three weeks, assuming "same code runs everywhere." When I deployed in week 18, Python 3.13 had broken `tflite-runtime`

entirely, and the IMX296 camera output BGR despite the API labelling it RGB888. These consumed two full days I could not afford [6]. The correction: I adopted a "day-one deployment" rule and created a Docker-based Pi environment test so model loading could be verified from my laptop. When weather cancelled our flight day, pre-planned bench tests kept the day productive.

5.3 Strengths and Weaknesses

My core strength is **rapid prototyping and system integration**—decomposing problems into bounded modules and getting a working system fast. But this is inseparable from my primary weakness: **speed over inclusion**. The same instinct that produced a working state machine in two days also meant I built it alone and presented teammates with a finished system rather than an opportunity to contribute. Edward observed that I "built things faster than anyone could review them"—meant as a compliment, but accurately describing the collaboration failure. The project manager noted feeling "out of the loop" on technical decisions, not from deliberate exclusion but because I moved faster than I communicated. I suspect this stems from Ukraine's engineering culture, where individual competence is prized above collaborative process. Learning to value slower, inclusive approaches as equally "productive" is an ongoing adjustment.

I communicate well through demonstrations and visual tools but am weaker at spontaneous verbal explanation. A second weakness is **delayed hardware testing**: I treated simulation as sufficient validation for too long. Platform-specific bugs (Python 3.13, camera colour, serial baud rates) are invisible in simulation by definition—a pattern I recognise from previous projects.

5.4 Development Programme

Based on Gibbs' reflective cycle [12] and Kolb's experiential learning model [10], I identify three development actions, with evidence of implementation already begun:

1. **Collaborative architecture from day one.** Define contribution points and interface contracts *before* implementation. Mid-project, I began applying this: `vision.py`'s single-function interface and numbered test scripts were deliberate attempts to lower the barrier—but these must exist from week 1.
2. **Day-one hardware deployment.** Run a trivial script on target hardware within the first week. Already implemented: after the Python 3.13 incident, I created a Docker Pi test and a preflight connectivity checker.
3. **Structured communication cadence.** Adopt a "one sentence, one diagram, one demo" framework and schedule regular check-ins rather than relying on teammates to ask.

Career goals. I came to Bristol from Ukraine to develop robotics skills I can bring back. The skills developed—MAVLink, edge AI, safety-critical design [5]—transfer directly to autonomous systems Ukraine needs for mine clearance, reconnaissance, and disaster response. This project's lesson: *the quality of an autonomous system is determined not by its algorithms' cleverness, but by the rigour of its testing and the honesty of its failure analysis.*

References

[1] Engineering Council. *The Accreditation of Higher Education Programmes (AHEP4)*. Tech. rep. https://www.engc.org.uk/ahep. Engineering Council UK, 2020.

[2] Raspberry Pi Foundation. *Raspberry Pi 5*. https://www.raspberrypi.com/products/raspberry-pi-5/. Accessed: 2026. 2023.
[3] Google. *TensorFlow Lite: Deploy Machine Learning Models on Mobile and Edge Devices*. https://www.tensorflow.org/lite. Accessed: 2026. 2023.
[4] D. A. Schön. *The Reflective Practitioner: How Professionals Think in Action*. Basic Books, 1983.
[5] N. G. Leveson. *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press, 2011.
[6] P. Koopman. *How Safe Is Safe Enough? Measuring and Predicting Autonomous Vehicle Safety*. Technical report. Carnegie Mellon University, 2019.
[7] M. Silvagni et al. "Multipurpose UAV for Search and Rescue Operations in Mountain Avalanche Events". In: *Geomatics, Natural Hazards and Risk* 8.1 (2017), pp. 18–33.
[8] M. A. Goodrich et al. "Supporting Wilderness Search and Rescue Using a Camera-Equipped Mini UAV". In: *Journal of Field Robotics*. Vol. 25. 2008, pp. 89–110.
[9] R. M. Belbin. *Team Roles at Work*. 2nd. Butterworth-Heinemann, 2010.
[10] D. A. Kolb. *Experiential Learning: Experience as the Source of Learning and Development*. Prentice-Hall, 1984.
[11] B. W. Tuckman. "Developmental Sequence in Small Groups". In: *Psychological Bulletin* 63.6 (1965), pp. 384–399.
[12] G. Gibbs. *Learning by Doing: A Guide to Teaching and Learning Methods*. Further Education Unit, Oxford Polytechnic, 1988.
[13] ArduPilot Development Team. *ArduPilot: Open Source Autopilot*. https://ardupilot.org/. Accessed: 2026. 2024.
[14] L. Meier, D. Honegger, and M. Pollefeys. *MAVLink: Micro Air Vehicle Communication Protocol*. https://mavlink.io/. Accessed: 2026. 2013.